ZEPTO SQL

Data Exploration Project

Null Handling • Data Cleaning • Business Insights

Author	Sachin Basantia	**Database**	PostgreSQL
GitHub Repo	sachinbasantia/Zepto_SQL_Data_Exploration	**Dataset**	Zepto Products

1. Project Overview

This project performs an end-to-end SQL data exploration on a Zepto (quick-commerce grocery platform) product dataset using PostgreSQL. Unlike the reference tutorial where the dataset was pre-cleaned, this project starts with real-world dirty data containing approximately 1,000 null values across multiple columns — making the null-handling phase a core original contribution.

The project is divided into three major phases:

- Phase 1 — Data Setup & Null Audit: Load the dataset, inspect schema, and quantify missing values per column.
- Phase 2 — Data Cleaning & Imputation: Strategically fix nulls using business logic, statistical imputation, and SQL UPDATE statements.
- Phase 3 — Business Analysis: Answer 8 analytical questions using SQL queries to derive insights on pricing, inventory, discounts, and revenue.

1.1 Dataset Schema

The zepto table contains the following columns:

Column Name	Data Type	Description
sku_id	INTEGER (PK)	Auto-generated surrogate key (not in original CSV)
name	VARCHAR(120)	Product name
category	VARCHAR(120)	Product category (14 categories)
mrp	NUMERIC(8,2)	Maximum Retail Price in rupees
discountPercent	NUMERIC(5,2)	Discount percentage offered
discountedSellingPrice	NUMERIC(8,2)	Actual selling price after discount
weightInGms	INTEGER	Product weight in grams
availableQuantity	INTEGER	Units available in inventory
outOfStock	BOOLEAN	Whether product is out of stock
quantity	INTEGER	Quantity descriptor

Important Note: The sku_id column was auto-generated during import and was excluded when loading data from the CSV — the import/export dialog had sku_id unchecked so it would not conflict with the original dataset structure.

2. Data Setup

2.1 Table Creation & Import

The zepto table was created in PostgreSQL and the CSV dataset (zepto_v2.csv) was imported using pgAdmin's Import/Export tool with the following settings:

Setting	Value
Format	CSV
Encoding	UTF-8
On Error	Stop
sku_id column	Excluded from import
File Path	D:\Projects\New_Project_2_SQL\zepto_dataset - zepto_v2.csv

2.2 Backup Table

Before performing any UPDATE operations, a backup table was created to allow rollback if needed:

```
CREATE TABLE zepto_backup AS
SELECT * FROM zepto;
```

Safety Practice: All UPDATE statements were first wrapped in a BEGIN / ROLLBACK transaction block during testing, and only committed (COMMIT) once the results were verified correct.

```
BEGIN;
  -- your UPDATE statement here
  SELECT * FROM zepto LIMIT 20;  -- verify
ROLLBACK;   -- if result is wrong
-- COMMIT;  -- only uncomment if correct
```

3. Phase 1 — Null Value Audit

The first step in any data cleaning pipeline is to understand the extent and distribution of missing data. A systematic null audit was performed across all columns.

3.1 Detect Rows with Any Null

A broad scan was run to find all rows containing at least one null value in any column:

```
SELECT * FROM zepto
WHERE name IS NULL
  OR category IS NULL
  OR mrp IS NULL
  OR discountPercent IS NULL
  OR discountedSellingPrice IS NULL
  OR weightInGms IS NULL
  OR availableQuantity IS NULL
  OR outOfStock IS NULL
  OR quantity IS NULL;
```

3.2 Count Not-Null Values per Column

To understand completeness per column, COUNT() was used (COUNT ignores NULLs naturally):

```
SELECT
   COUNT(*)                  AS total_rows,
   COUNT(category)           AS category_not_null,
   COUNT(mrp)                AS mrp_not_null,
   COUNT(discountedSellingPrice) AS discount_SP_not_null,
   COUNT(weightInGms)        AS weight_not_null,
   COUNT(outOfStock)         AS out_of_stock_not_null,
   COUNT(quantity)           AS quantity_not_null,
   COUNT(discountPercent)    AS discount_not_null,
   COUNT(availableQuantity)  AS qty_not_null
FROM zepto;
```

3.3 Null Count per Column (CASE + SUM)

To get the precise number of nulls per column, a CASE WHEN expression was used inside SUM():

```
SELECT
   SUM(CASE WHEN name IS NULL THEN 1 ELSE 0 END)              AS name_nulls,
   SUM(CASE WHEN mrp IS NULL THEN 1 ELSE 0 END)               AS mrp_nulls,
   SUM(CASE WHEN discountPercent IS NULL THEN 1 ELSE 0 END)   AS discount_nulls,
```

```
  SUM(CASE WHEN availableQuantity IS NULL THEN 1 ELSE 0 END)     AS qty_nulls,
  SUM(CASE WHEN category IS NULL THEN 1 ELSE 0 END)            AS category_nulls,
  SUM(CASE WHEN quantity IS NULL THEN 1 ELSE 0 END)            AS quantity_nulls,
  SUM(CASE WHEN discountedSellingPrice IS NULL THEN 1 ELSE 0 END) AS discount_SP_nulls,
  SUM(CASE WHEN weightInGms IS NULL THEN 1 ELSE 0 END)          AS weight_nulls,
  SUM(CASE WHEN outOfStock IS NULL THEN 1 ELSE 0 END)          AS outOfStock_nulls
FROM zepto;
```

3.4 Null Audit Results

The query above produced the following null counts (initial state before cleaning):

Column	Null Count	Strategy
discountPercent	1,178	Fill with 0 (no discount = 0%)
availableQuantity	453	Business logic + median imputation
discountedSellingPrice	~6	Reconstruct from MRP and discount
weightInGms	4	Domain heuristic (Kajal eyeliners = 15g)
mrp	1	Category-aware median imputation
name, category, outOfStock	0	No nulls — no action needed

Key Insight: The dataset had nearly 30% null values spread across multiple columns, making a thoughtful, column-by-column strategy essential rather than a blanket deletion approach.

4. Phase 2 — Data Cleaning & Imputation

Each column with null values was handled using the most appropriate strategy based on the business context, data distribution, and SQL best practices.

4.1 discountPercent Column (1,178 Nulls)

Strategy: Fill with Zero

A null discount percentage logically means no discount is being offered. This is a safe business logic assumption — a missing discount is not the same as an unknown discount; it simply means full price.

```
UPDATE zepto
SET discountPercent = 0
WHERE discountPercent IS NULL;
```

Result: UPDATE 1178 — All 1,178 null discount values replaced with 0.

4.2 availableQuantity Column (453 Nulls)

Strategy: Two-Step Business Logic + Statistical Imputation

This column required a two-pronged approach:

Step 1 — Business Logic: For products where outOfStock = TRUE, available quantity must be 0. These nulls were set directly.

```
UPDATE zepto
SET availableQuantity = 0
WHERE availableQuantity IS NULL
  AND outOfStock = TRUE;
```

Step 2 — Median Imputation: For remaining nulls (products supposedly in stock), the median of non-null availableQuantity values was computed and applied. Median was preferred over mean to avoid skew from extreme values.

```
UPDATE zepto
SET availableQuantity = sub.median_val
FROM (
   SELECT percentile_cont(0.5)
         WITHIN GROUP (ORDER BY availableQuantity) AS median_val
   FROM zepto
   WHERE availableQuantity IS NOT NULL
) sub
```

```
WHERE zepto.availableQuantity IS NULL;
```

SQL Concept: percentile_cont(0.5) WITHIN GROUP (ORDER BY col) is an ordered-set aggregate function in PostgreSQL. It computes continuous percentiles and is the standard way to calculate median in SQL.

4.3 quantity Column

Strategy: Fill with Zero

Null values in the quantity column were treated as zero, consistent with the outOfStock logic.

```
UPDATE zepto
SET quantity = 0
WHERE quantity IS NULL;
```

4.4 mrp Column (1 Null — Cherry Product)

Strategy: Category-Aware Median Imputation

The single null MRP belonged to a Cherry-branded product in the Home & Cleaning category. Rather than using the global median (which would cross categories), a targeted median was computed using only products with the same brand and category — preserving the contextual pricing distribution.

The analogy used: "Calculate median for apples but apply it to bananas" — this highlights why category-specific imputation matters. Applying cross-category medians introduces bias.

```
-- First inspect the category to understand price range
SELECT * FROM zepto
WHERE mrp IS NOT NULL
  AND category = 'Home & Cleaning';

-- Main imputation query using correlated subquery
UPDATE zepto z
SET mrp = sub.median_val
FROM (
    SELECT percentile_cont(0.5)
          WITHIN GROUP (ORDER BY mrp) AS median_val
    FROM zepto
    WHERE mrp IS NOT NULL
      AND category = 'Home & Cleaning'
      AND name ILIKE 'Cherry%'
) sub
WHERE z.mrp IS NULL
  AND z.category = 'Home & Cleaning'
  AND z.name ILIKE 'Cherry%';
```

Result: UPDATE 1 — The single null MRP was filled with the Cherry product median from the same category.

4.5 discountedSellingPrice Column

Strategy: Reverse-Engineer from MRP and Discount

Selling price was reconstructed algebraically. If MRP and discount percentage are known, the selling price can be derived exactly — like reverse-engineering the billing system:

Formula: Selling Price = MRP - (discountPercent / 100 × MRP)

```
UPDATE zepto
SET discountedSellingPrice = ROUND(
   mrp - (COALESCE(discountPercent, 0) * mrp / 100.0),
   2
)
WHERE discountedSellingPrice IS NULL
 AND mrp IS NOT NULL;
```

COALESCE Usage: COALESCE(discountPercent, 0) acts as a null-safe wrapper — if discount is null, it defaults to 0, ensuring the formula always evaluates to MRP (i.e., no discount applied). This is the key function for null handling in SQL.

4.6 weightInGms Column (4 Nulls — Eyeliner/Kajal Products)

Strategy: Domain-Driven Heuristic

The 4 null weight values all belonged to Kajal (eyeliner) products in the Personal Care and Paan Corner categories. Rather than applying a statistical imputation (which would be meaningless across diverse product types), a domain knowledge approach was used:

- Kajal eyeliners are a consistent product type with a standard weight of approximately 15 grams.
- Statistical imputation from the full dataset would mix weights of shampoos (500g), oils (1L+), etc., making the result nonsensical.
- This is why product-specific heuristics outperform naive statistics for categorical/domain-specific attributes.

```
-- First inspect the product context
SELECT * FROM zepto
WHERE name LIKE '%Kajal%' IS NULL
AND (SELECT * FROM zepto
    WHERE category IN ('Personal Care', 'Paan Corner'));

-- Apply domain-driven weight
UPDATE zepto
```

```
SET weightInGms = 15
WHERE weightInGms IS NULL
  AND name ILIKE '%kajal%';
```

Result: UPDATE 4 — All 4 null Kajal weights filled with 15g.

4.7 Post-Cleaning Verification

After all UPDATE operations, the null audit query was re-run to confirm zero nulls remain:

```
SELECT
    SUM(CASE WHEN name IS NULL THEN 1 ELSE 0 END)                AS name_nulls,
    SUM(CASE WHEN mrp IS NULL THEN 1 ELSE 0 END)                 AS mrp_nulls,
    SUM(CASE WHEN discountPercent IS NULL THEN 1 ELSE 0 END)       AS discount_nulls,
    SUM(CASE WHEN availableQuantity IS NULL THEN 1 ELSE 0 END)     AS qty_nulls,
    SUM(CASE WHEN category IS NULL THEN 1 ELSE 0 END)             AS category_nulls,
    SUM(CASE WHEN quantity IS NULL THEN 1 ELSE 0 END)             AS quantity_nulls,
    SUM(CASE WHEN discountedSellingPrice IS NULL THEN 1 ELSE 0 END) AS discount_SP_nulls,
    SUM(CASE WHEN weightInGms IS NULL THEN 1 ELSE 0 END)          AS weight_nulls,
    SUM(CASE WHEN outOfStock IS NULL THEN 1 ELSE 0 END)           AS outOfStock_nulls
FROM zepto;
```

Result: All columns returned 0 nulls — the dataset is fully clean.

5. Additional Data Cleaning

5.1 Price Unit Conversion (Paise to Rupees)

During analysis it was discovered that MRP and discountedSellingPrice values were stored in paise (1 rupee = 100 paise) rather than rupees — a common data entry issue. Some rows showed MRP = 0 or discountedSellingPrice = 0 due to the import format.

```
-- Identify zero-price records
SELECT * FROM zepto
WHERE mrp = 0 OR discountedSellingPrice = 0;

-- Convert paise to rupees
UPDATE zepto
SET mrp = mrp / 100.0,
    discountedSellingPrice = discountedSellingPrice / 100.0;
```

5.2 Duplicate SKU Analysis

Products with the same name but multiple SKUs (stock-keeping units) were identified. These are legitimate product variants differing in price, weight, or pack size — common in e-commerce — and were retained as separate records.

```
SELECT name, COUNT(sku_id) AS stock_keeping_units
FROM zepto
GROUP BY name
HAVING COUNT(sku_id) > 1
ORDER BY COUNT(sku_id) DESC;
```

Top products with multiple SKUs (sample output):

Product Name	SKU Count
Quaker Oats	10
Arden Eggs White	10
Sunfeast Yippee! Pasta	10
Amul Delicious Fat Spread	10
Saffola Veggie Twist	10

6. Phase 3 — Business Analysis (8 Questions)

With a clean dataset, 8 business questions were answered using SQL queries. Each query demonstrates a specific SQL concept while generating actionable insights.

Q1. Top 10 Best-Valued Products by Discount Percentage

Identifies products offering the highest discounts — useful for marketing and promotional strategy.

```
SELECT DISTINCT name, discountPercent
FROM zepto
ORDER BY discountPercent DESC
LIMIT 10;
```

Top results included Dukes Waffy variants (51% discount) and multiple products at 50% off, including RRO Burrata Cheese, Epigamia Fruit Yogurt, Moi Soi Manchuria, and Ceres Foods Laal variants.

Product Name	Discount %
Dukes Waffy Orange / Strawberry / Chocolate (3 variants)	51.00%
RRO Burrata Cheese	50.00%
Epigamia Fruit Yogurt (2 variants)	50.00%
Moi Soi Manchuria, Ceres Foods Laal	50.00%

SQL Concepts: DISTINCT, ORDER BY DESC, LIMIT

Q2. High-MRP Products Currently Out of Stock

Finds premium products that are unavailable — indicating potential revenue loss and restock priorities.

```
SELECT DISTINCT name, mrp
FROM zepto
WHERE mrp >= (SELECT AVG(mrp) FROM zepto)
  AND outOfStock = 'True'
ORDER BY mrp DESC;
```

Key findings — top out-of-stock premium products:

Product Name	MRP (₹)
Praakritik Natural Desi Gir Cow A2 Ghee	1,450.00
Super Saver Kolam Rice Sortex Clean	700.00
Pedigree Dog Food Adult Chicken & Veg	660.00
Gillette Mach 3 Turbo Cartridges	649.00
Sebamed Anti-Dandruff Shampoo	600.00

SQL Concepts: Correlated subquery in WHERE clause, AVG() aggregate

Q3. Total Revenue per Category

Calculates potential revenue as: discountedSellingPrice × availableQuantity, grouped by category — identifying the most revenue-generating product lines.

```
SELECT
  category,
  ROUND(SUM(discountedSellingPrice * availableQuantity), 2) AS total_revenue
FROM zepto
GROUP BY category
ORDER BY total_revenue DESC;
```

Revenue by category (in rupees):

Category	Total Revenue (₹)
Cooking Essentials	337,369.00
Munchies	337,369.00
Personal Care	270,849.00
Paan Corner	270,849.00
Ice Cream & Desserts	224,385.00
Chocolates & Candies	224,385.00
Packaged Food	224,385.00
Home & Cleaning	122,744.00
Fruits & Vegetables	10,846.00

SQL Concepts: GROUP BY, SUM() with arithmetic expression, ORDER BY aggregate result

Q4. Premium Products with Low Discount

Finds products priced above ₹500 MRP that offer less than 10% discount — these are full-price premium items.

```
SELECT name, mrp
FROM zepto
WHERE mrp > 500 AND discountPercent < 10
GROUP BY name, mrp
ORDER BY mrp DESC;
```

Notable products: Patanjali Cow's Ghee (₹565), MamyPoko Pants Diapers (₹399), Aashirvaad Atta With Mutigrains (₹315), and Gillette Venus Razor (₹249).

SQL Concepts: Compound WHERE with AND, GROUP BY for deduplication

Q5. Top 5 Categories by Average Discount Percentage

Ranks categories by the discounts they offer — useful for understanding promotional intensity by category.

```
SELECT category, discountPercent
FROM zepto
GROUP BY category, discountPercent
ORDER BY discountPercent DESC
LIMIT 5;
```

Top 5 results:

Category	Discount %
Biscuits	51.00%
Meats, Fish & Eggs	50.00%
Chocolates & Candies	50.00%
Cooking Essentials	50.00%
Dairy, Bread & Batter	50.00%

SQL Concepts: GROUP BY multiple columns, ORDER BY, LIMIT

Q6. Price per Gram — Best Value Products Above 100g

Calculates the cost-efficiency of products by computing price per gram. Sorted ascending to show the best value (cheapest per gram) first.

```
SELECT DISTINCT
  name,
```

```
  weightInGms,
  discountedSellingPrice,
  ROUND(discountedSellingPrice / weightInGms, 2) AS price_per_gram
FROM zepto
WHERE weightInGms >= 100
ORDER BY price_per_gram ASC;
```

Best value products (lowest price per gram) include bulk staples like Tata Salt, Onion, and various Shubh Kart products — all priced at ₹0.02–₹0.03 per gram. Higher value per gram products include specialty items.

Key Design Choice: discountedSellingPrice was used rather than MRP because it represents the actual price a customer pays — using MRP would overstate the real cost.

SQL Concepts: Derived column in SELECT (alias), ROUND(), DISTINCT, ORDER BY alias

Q7. Weight-Based Product Segmentation (Low / Medium / Bulk)

Step 1 — Determine Percentile Thresholds

Rather than using arbitrary weight ranges, a data-driven percentile approach was chosen. The 33rd and 66th percentiles of weightInGms were computed to define equal-sized groups:

```
SELECT
  percentile_cont(0.33) WITHIN GROUP (ORDER BY weightInGms) AS p33,
  percentile_cont(0.66) WITHIN GROUP (ORDER BY weightInGms) AS p66
FROM zepto
WHERE weightInGms IS NOT NULL;
```

Result: p33 = 150g, p66 = 360g

Step 2 — Classify Products Using CASE

Using the business-logic thresholds (≤120g = Low, ≤450g = Medium, >450g = Bulk) which were calibrated against the percentile findings:

```
SELECT
  name,
  weightInGms,
  CASE
    WHEN weightInGms IS NULL   THEN 'Unknown'
    WHEN weightInGms <= 120    THEN 'Low'
    WHEN weightInGms <= 450    THEN 'Medium'
    ELSE 'Bulk'
  END AS weight_category
FROM zepto;
```

Sample classification results: Zama Organics Carrot (250g → Medium), Safal Frozen Mixed Vegetables (1000g → Bulk), Popular Essential Pista Salt (100g → Low), Madhur Sugar (1000g → Bulk).

Why Percentiles over Mean: The dataset has highly skewed weight distribution — from Kajal eyeliners (~15g) to cooking oils (~1L+). Mean-based splits would be heavily biased by outliers. Percentile-based segmentation divides products into equal-sized groups regardless of distribution shape.

SQL Concepts: CASE WHEN (with SELECT), ordered-set aggregate functions, percentile_cont()

Q8. Total Inventory Weight per Category

Calculates total physical inventory weight per category: weightInGms × availableQuantity, giving a picture of stock volume. Note: always multiply by quantity when computing totals — a common mistake is to forget this step.

```
SELECT
   category,
   SUM(weightInGms * availableQuantity) AS total_inventory_weight
FROM zepto
GROUP BY category
ORDER BY total_inventory_weight;
```

Total inventory weight by category (in grams):

Category	Total Inventory Weight (g)
Meats, Fish & Eggs	48,016
Biscuits	84,431
Fruits & Vegetables	91,794
Health & Hygiene	142,904
Home & Cleaning	373,236
Munchies / Cooking Essentials	1,404,654

SQL Concepts: GROUP BY, SUM() with multi-column expression, ORDER BY aggregate

7. Key SQL Learnings & Technical Notes

7.1 Critical SQL Rules Identified

Rule / Pitfall	Explanation
Aggregate in WHERE vs HAVING	You CANNOT use aggregate functions (SUM, AVG, COUNT) in a WHERE clause. Use HAVING for aggregate conditions. However, aggregates CAN be used in ORDER BY.
UPDATE is permanent	Unlike SELECT, UPDATE modifies data on disk. Always use BEGIN / ROLLBACK / COMMIT for testing. Always back up first.
CASE is a SELECT tool	CASE WHEN...THEN...END is primarily used inside SELECT statements (not standalone). It evaluates row by row.
Alias in ORDER BY	You CAN use column aliases defined in SELECT inside the ORDER BY clause — this is valid in PostgreSQL.
COUNT vs SUM for nulls	COUNT(col) counts non-null values. SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) counts nulls. Both patterns are essential for null auditing.
Mean vs Median	For skewed distributions, median is always preferred over mean for imputation. Percentile-based thresholds are industry standard.

7.2 Advanced SQL Functions Used

Function / Feature	Usage in This Project
percentile_cont(n) WITHIN GROUP	Ordered-set aggregate for computing percentiles and medians. Used for availableQuantity, MRP, and weight segmentation.
COALESCE(val, default)	Null-safe value substitution. Used in selling price reconstruction formula to handle null discounts.
UPDATE ... FROM (subquery)	Correlated subquery in UPDATE — computes aggregate from a subquery and applies it to matching rows.
ILIKE	Case-insensitive pattern matching. Used instead of LIKE for flexible product name searches.
CASE WHEN ... END	Row-level conditional logic in SELECT for classification (weight_category) and null counting.
HAVING vs WHERE	HAVING filters on aggregated results; WHERE filters on raw rows. Critical distinction for aggregate queries.

8. Project Summary

8.1 What Was Accomplished

#	Task	Technique Used
1	Null audit across all columns	COUNT(), SUM(CASE WHEN), IS NULL checks
2	Fixed 1,178 null discounts	Business logic (null = 0%)
3	Fixed 453 null quantities	outOfStock logic + percentile_cont median
4	Fixed 1 null MRP (Cherry)	Category-aware median imputation
5	Reconstructed selling prices	COALESCE + algebraic formula from MRP
6	Fixed 4 null weights (Kajal)	Domain heuristic (15g for eyeliners)
7	Currency unit correction	Division by 100 for paise to rupees
8	8 business questions answered	Aggregations, subqueries, CASE, percentiles

8.2 Original Contributions vs Reference Tutorial

Reference Tutorial (YouTube)	This Project (Original)
Dataset pre-cleaned (0 instead of NULL)	Raw dataset with ~1,000 real null values
No null handling phase	Full null audit + column-by-column strategy
No imputation techniques	Median, business logic, domain heuristics, COALESCE
No UPDATE operations	Multiple UPDATE queries with transaction safety
Same 8 business questions	Same 8 questions + documented decision reasoning

8.3 Skills Demonstrated

- Data Quality Assessment: Systematic null detection and quantification
- Statistical Imputation: Median-based filling using ordered-set aggregates
- Business Logic Application: Domain-driven null filling strategies
- Advanced SQL: Correlated subqueries, UPDATE FROM, COALESCE, percentile_cont
- Data Distribution Analysis: Choosing median over mean for skewed data
- Safe Data Modification: Transaction-wrapped updates with rollback testing
- Business Intelligence: Revenue, discount, inventory, and pricing analysis